

ALERTMIND

AI-Assisted Mini Security Operations Centre

Project Report

IIT Roorkee × FutureNSE

PG Certificate Programme in AI and GenAI-Powered Cybersecurity

Team: Brain Bytes

Indrajit Singh • Jay Goyal • Vansh Gupta

Repository:

<https://github.com/indrajitisingh/AlertMind/tree/main>

Table of Contents

1. Executive Summary
2. Introduction
3. Problem Statement and Motivation
4. Project Objectives
5. System Architecture
6. AI Processing Pipeline and Guardrails
7. Technology Stack
8. Custom Detection Engineering
9. Dashboards and Visualization
10. Incident Response Playbook
11. MTTD and MTTR Evaluation
12. Responsible AI Design
13. Results and Discussion
14. Limitations and Future Work
15. Conclusion
16. Team Contributions
17. References and Repository

1. Executive Summary

AlertMind is an AI-assisted Mini Security Operations Centre (SOC) developed as a capstone project for the IIT Roorkee × Futureense PG Certificate Programme in AI and GenAI-Powered Cybersecurity. The project integrates an open-source SIEM (Wazuh), an OpenSearch-backed alert index, Sysmon-based endpoint telemetry, and a custom Streamlit dashboard that layers AI-assisted triage on top of raw security alerts.

The platform detects a set of Windows endpoint reconnaissance and execution techniques — including system enumeration, network discovery, and suspicious command execution — using thirteen custom Wazuh detection rules (100101–100113). When an analyst reviews a generated alert, AlertMind's AI Assistant, built on OpenAI's GPT-4o-mini, produces a structured, human-readable analysis: an executive summary, a MITRE ATT&CK mapping, a severity classification, extracted indicators of compromise (IOCs), recommended investigation steps, and suggested incident response actions drawn from the project's own Incident Response Playbook.

Crucially, AlertMind does not perform detection itself and does not act autonomously. Wazuh remains solely responsible for detecting malicious activity; the AI Assistant operates strictly downstream of detection as a decision-support layer, and every recommendation it produces is reviewed and validated by a human analyst before any response action is taken. This report consolidates the project's architecture, detection engineering, AI pipeline design, guardrail mechanisms, dashboarding, incident response documentation, and workflow-efficiency evaluation into a single reference document.

2. Introduction

Modern Security Operations Centres face a persistent challenge: alert volume routinely exceeds the capacity of available analysts. Each alert generated by a SIEM typically requires a human to read raw log data, identify the relevant MITRE ATT&CK technique, and locate the corresponding incident response procedure before any containment action can begin. This manual triage process is accurate but slow, and it scales poorly as alert volume grows or as analyst experience varies across a team.

AlertMind was conceived to address this bottleneck without displacing the human analyst or the underlying detection engine. The project pairs a conventional, rule-based SIEM (Wazuh) — which remains fully responsible for detecting suspicious activity — with a lightweight AI assistant that helps analysts interpret alerts faster once they have already been raised. The result is a small but complete SOC pipeline: endpoint telemetry collection, centralized detection, alert indexing, AI-assisted triage, and a documented, rule-mapped incident response process.

3. Problem Statement and Motivation

SOC analysts, particularly less experienced ones, spend a disproportionate share of their time on the investigation phase of incident response rather than on the response itself. For each alert, an analyst must typically: (1) read raw JSON event data containing fields such as command lines, parent processes, and file hashes; (2) determine which MITRE ATT&CK technique the behavior corresponds to; and (3) locate the relevant section of an incident response playbook to identify the correct containment and remediation steps. Each of these steps introduces delay, and the cumulative effect is a longer Mean Time to Respond (MTTR) even when detection itself (Mean Time to Detect, or MTTD) is near-instantaneous.

AlertMind's motivation is to demonstrate, at capstone scale, how a constrained and guarded AI assistant can meaningfully shorten the investigation phase of this workflow — summarizing alerts, mapping techniques, and recommending next steps — while leaving detection, validation, and response authority entirely with the human analyst and the SIEM.

4. Project Objectives

- Deploy a functional Wazuh SIEM environment with a monitored Windows endpoint and Sysmon-based telemetry collection.
- Engineer a custom set of Wazuh detection rules covering representative endpoint reconnaissance and execution techniques.
- Design and build a Streamlit-based SOC dashboard (AlertMind) capable of retrieving alerts from OpenSearch and accepting manual alert input.
- Integrate an AI analysis engine that summarizes alerts, maps MITRE ATT&CK techniques, classifies severity, extracts IOCs, and recommends investigation and response steps.
- Implement guardrails that validate and sanitize alert data before it is ever submitted to an external AI model.
- Document a rule-mapped Incident Response Playbook covering the detection categories implemented in the project.
- Evaluate the qualitative impact of AI-assisted triage on analyst workflow (MTTD/MTTR) relative to a manual baseline.

5. System Architecture

AlertMind is structured as a linear, six-layer pipeline: telemetry is collected on the monitored Windows endpoint, forwarded to a centralized Wazuh Manager, indexed in OpenSearch, retrieved and enriched by the AlertMind Streamlit dashboard, analyzed by OpenAI's GPT-4o-mini, and finally reviewed by a human SOC analyst. Detection occurs entirely within Wazuh; every layer downstream of OpenSearch is concerned with presenting and interpreting alerts rather than generating them.



Figure 5.1: AlertMind end-to-end system architecture, from endpoint telemetry to analyst review.

5.1 Component Overview

- **Windows Endpoint** (Windows 11 / Windows 10) — runs the Wazuh Agent and Sysmon, generating process-creation and command-line telemetry.
- **Ubuntu Server** — hosts the Wazuh Manager and Wazuh API, which receive agent telemetry, apply detection rules, and generate alerts.
- **OpenSearch** — stores and indexes all Wazuh alerts, providing the query interface used by both the Wazuh dashboard and AlertMind.
- **AlertMind** (Streamlit Dashboard) — retrieves alerts (live fetch or manual JSON paste) and orchestrates AI-assisted analysis: MITRE ATT&CK mapping, IOC extraction, investigation guidance, and IR recommendations.
- **OpenAI GPT-4o-mini** — the language model used to generate structured security intelligence from a sanitized alert payload.
- **SOC Analyst** — reviews AI-generated findings alongside the original alert and decides on and executes the response.

6. AI Processing Pipeline and Guardrails

Once an alert reaches AlertMind, it passes through a nine-stage internal pipeline before any output is shown to the analyst. The pipeline is deliberately linear and auditable: every alert is parsed, validated, and sanitized before it is ever transmitted to the external AI model, and the model's response is structured into discrete, labeled outputs rather than free-form text.



Figure 6.1: AlertMind's internal AI processing pipeline, from raw alert to analyst-facing output.

6.1 Guardrails Engine

Before any alert data leaves the AlertMind application, it passes through a Guardrails Engine (implemented in `guardrails.py`) that performs four checks:

- **Payload Size Validation** — rejects abnormally large payloads before processing.
- **Wazuh Schema Validation** — confirms the submitted JSON matches the expected Wazuh alert structure.
- **Prompt Injection Detection** — screens alert content for text patterns designed to manipulate the AI model's instructions.
- **Sensitive Data Redaction** — strips or masks sensitive fields prior to submission to the external AI provider.

Only after passing all four checks is a sanitized payload submitted to GPT-4o-mini. The model's response is required to be structured JSON, which AlertMind then renders as six discrete outputs: AI Summary, MITRE ATT&CK Mapping, Severity Classification, IOC Extraction, Investigation Steps, and Incident Response Actions.

7. Technology Stack

Layer	Technology	Purpose
Endpoint	Windows 10/11, Sysmon, Wazuh Agent	Telemetry generation and forwarding
Detection	Wazuh Manager, Wazuh API (Ubuntu)	Rule-based event correlation and alerting
Storage/Index	OpenSearch	Alert indexing and query interface
Application	Python, Streamlit	AlertMind dashboard and orchestration
AI Analysis	OpenAI GPT-4o-mini	Structured security intelligence generation
Application Modules	app.py, analyzer.py, guardrails.py, prompts.py, styles.py, utils.py, opensearch_client.py, wazuh_api.py	Seven-module Python package implementing the dashboard

8. Custom Detection Engineering

AlertMind implements thirteen custom Wazuh detection rules (rule IDs 100101–100113), each mapped to a specific Windows command commonly associated with post-exploitation reconnaissance or living-off-the-land execution. The rules span three categories used consistently throughout the project's documentation: System Enumeration, Network Discovery, and Suspicious Command Execution.

Rule ID	Command	Description
100101	whoami	User Information Discovery
100102	ipconfig	Network Configuration Discovery
100103	hostname	System Information Discovery
100107	net localgroup	Local Group Discovery
100108	netstat	Network Connections Discovery
100109	ping	Network Discovery
100110	PowerShell	PowerShell Execution
100111	cmd	Command Prompt Execution
100112	certutil	CertUtil Execution
100113	nslookup	DNS Lookup

8.1 Detection Evidence

The following captures illustrate representative detections from the monitored endpoint (DESKTOP-M95IQ0B), each showing the Wazuh Threat Hunting view with the triggering rule, timestamp, and rule description.

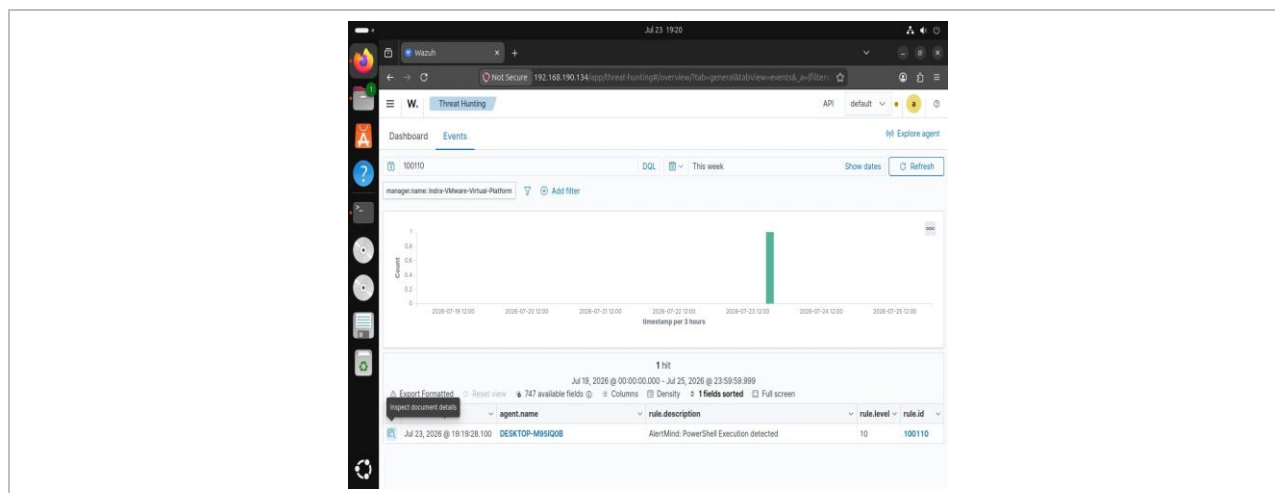


Figure 8.1: Rule 100110 — PowerShell Execution detected.

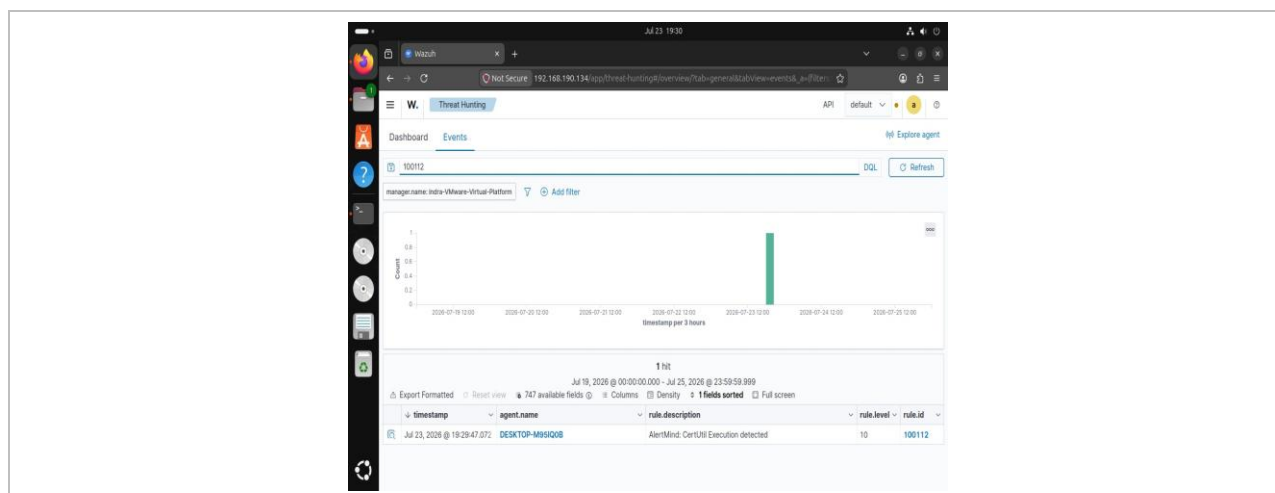


Figure 8.2: Rule 100112 — CertUtil Execution detected.

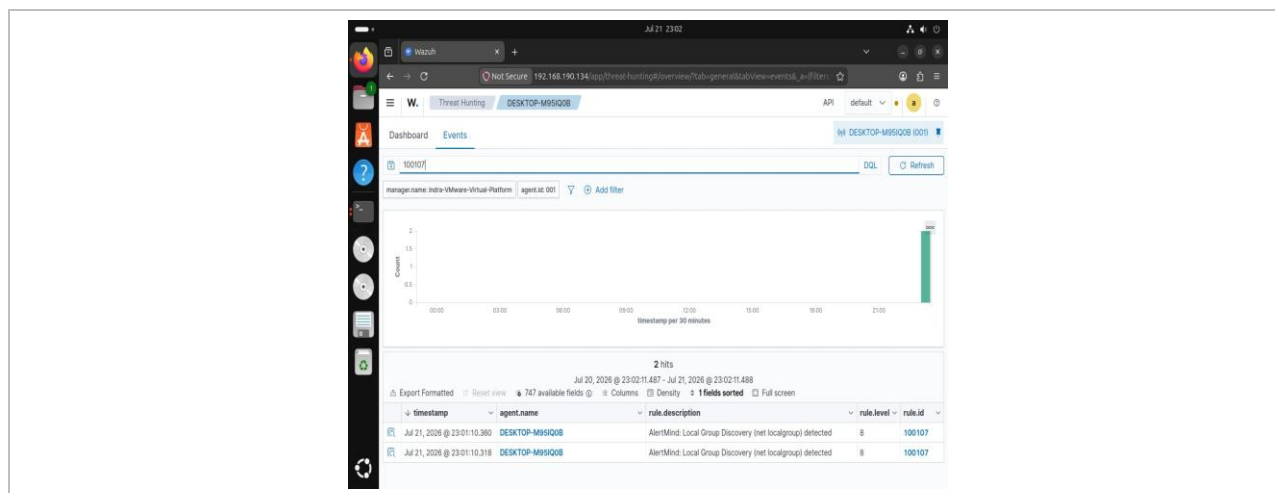


Figure 8.3: Rule 100107 — Local Group Discovery (net localgroup) detected.

9. Dashboards and Visualization

In addition to AlertMind's own AI-assisted triage view, the underlying Wazuh platform provides operational dashboards used for day-to-day monitoring and MITRE ATT&CK trend analysis. Three representative views are included below.

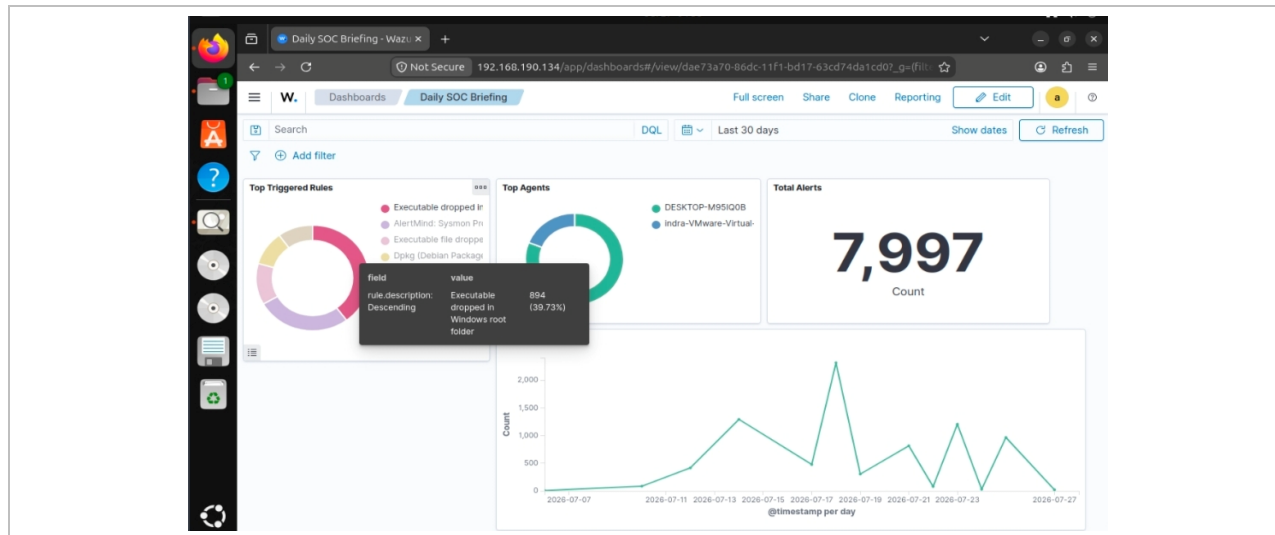


Figure 9.1: Daily SOC Briefing dashboard — Top Triggered Rules, Top Agents, and Total Alerts (7,997) over the last 30 days.

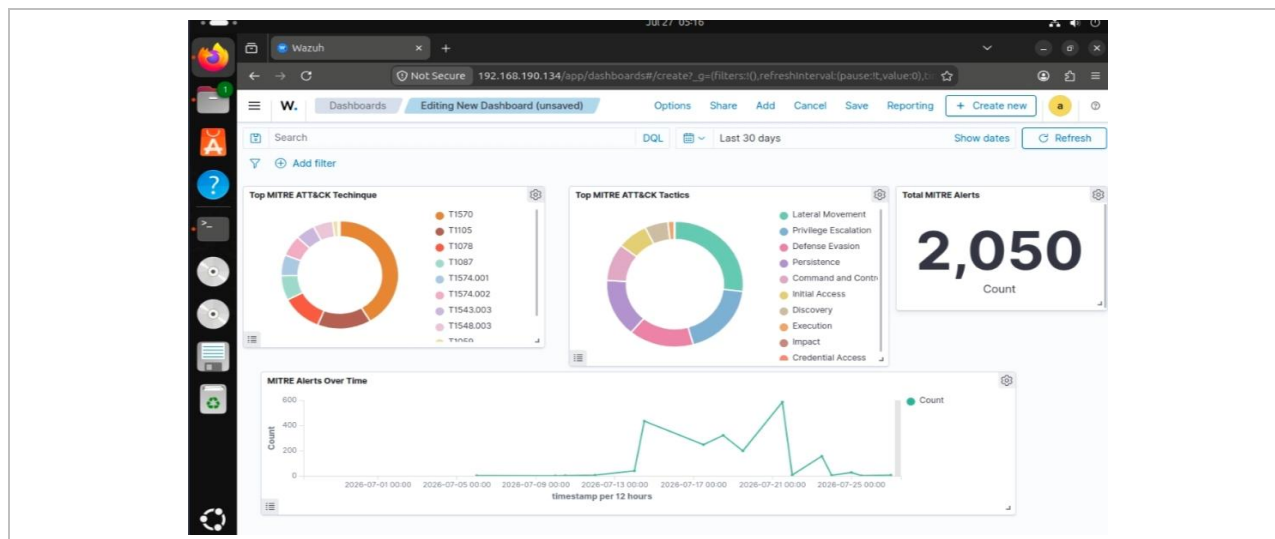


Figure 9.2: MITRE ATT&CK dashboard — Top techniques, top tactics, and total MITRE-mapped alerts (2,050).

Figure 9.3: Endpoint summary for DESKTOP-M95IQ0B — event count evolution, top MITRE ATT&CK tactics, and PCI DSS compliance posture.

10. Incident Response Playbook

AlertMind is accompanied by a dedicated Incident Response Playbook covering the three detection categories implemented in this project: System Enumeration, Network Discovery, and Suspicious Command Execution. Each category follows a standard seven-part incident lifecycle — Incident Overview, Detection, Analysis, Containment, Eradication, Recovery, and Lessons Learned — and is directly mapped to the custom rule IDs in Section 8.1, with supporting evidence screenshots from the AlertMind/Wazuh implementation.

The playbook is maintained as a standalone document (IR_Playbook.docx) in the project repository and is referenced by AlertMind's AI Assistant when it recommends an incident response action, closing the loop between AI-generated guidance and documented analyst procedure.

11. MTTD and MTTR Evaluation

To characterize AlertMind's impact on analyst workflow, a lightweight before/after comparison was conducted between a manual triage baseline and the AI-assisted workflow. Because Wazuh performs detection independently of the AI assistant, Mean Time to Detect (MTTD) is unaffected by AlertMind and remains near real-time — typically within seconds to approximately one minute, depending on log ingestion and indexing latency. The measurable difference appears in Mean Time to Respond (MTTR), where AI-assisted summarization, MITRE ATT&CK mapping, and playbook recommendation shorten the analyst's path from alert to informed decision.

Metric	Without AI (Baseline)	With AlertMind AI
Detection Method	Wazuh	Wazuh
Alert Analysis	Manual (raw JSON/log review)	AI-assisted (structured summary)
MITRE ATT&CK Mapping	Manual	Automatic (analyst-validated)
IR Playbook Selection	Manual	AI-recommended
MTTD	Near real-time (seconds to ~1 min)	Near real-time (seconds to ~1 min)
MTTR	~4–6 minutes (representative estimate)	~1–1.5 minutes (representative measurement)

These figures are representative workflow measurements collected in a controlled lab environment rather than the output of a formal, repeated-trial benchmark study, and are documented in full — including methodology and threats to validity — in the project's standalone MTTD & MTTR Analysis report.

12. Responsible AI Design

AlertMind's design deliberately limits the AI model's role and authority within the SOC workflow:

- Detection independence — Wazuh detects all activity; the AI model never sees raw endpoint telemetry until after an alert already exists.
- Sanitization before submission — every alert passes through the Guardrails Engine's four checks before reaching the AI model.
- Structured, bounded output — the model is constrained to return structured JSON across six defined fields rather than open-ended text.
- Human validation — all AI-generated findings are explicitly presented as recommendations for the analyst to review, not as automated actions.
- No autonomous response — AlertMind does not trigger containment, blocking, or remediation actions on its own; every response action is executed by the human analyst.

This design reflects a decision-support rather than decision-making model of AI integration, consistent with responsible practice for AI use in security-critical environments.

13. Results and Discussion

Across the testing period, the monitored endpoint generated 7,997 total alerts over a 30-day window, with 2,050 alerts successfully mapped to MITRE ATT&CK techniques and tactics spanning Initial Access, Discovery, Persistence, Privilege Escalation, Defense Evasion, and Command and Control. The thirteen custom detection rules reliably triggered against their corresponding simulated commands, and the AI Assistant consistently returned structured, technique-mapped analysis within the sub-two-minute window documented in Section 11.

These results support the project's central thesis: AlertMind's primary value lies in reducing the analyst's investigation burden rather than in accelerating detection itself. Detection speed and accuracy remain governed entirely by Wazuh's rule engine; AlertMind's contribution is a faster, more consistent path from a raised alert to an informed, technique-mapped response decision.

14. Limitations and Future Work

- Testing was conducted in a single-endpoint lab environment rather than a multi-host production SOC; results may not generalize directly to higher alert volumes or more heterogeneous infrastructure.
- The current rule set (100101–100113) covers reconnaissance and execution techniques; expanding coverage to lateral movement, persistence, and exfiltration techniques is a natural next step.
- MTTR figures are representative estimates rather than the output of a formal, statistically controlled benchmark; a larger-scale, multi-analyst timed study would strengthen these findings.
- Background polling and immediate JWT token refresh were deprioritized for the demo build and would be required for continuous production use.
- Future iterations could extend AlertMind's guardrails with rate limiting and per-analyst audit logging of AI-assisted decisions.

15. Conclusion

AlertMind demonstrates a practical, constrained approach to integrating AI into SOC operations: a conventional SIEM continues to perform all detection, while a guarded, structured AI assistant accelerates the analyst's investigation and decision-making process. The project delivers a complete, working pipeline — from endpoint telemetry to custom detection rules, AI-assisted triage, a documented Incident Response Playbook, and a workflow-efficiency evaluation — and illustrates how AI can meaningfully support, without replacing, human judgment and accountability in incident response.

16. Team Contributions

This project was completed by Team Brain Bytes. Primary areas of contribution are summarized below.

Name	Primary Area of Contribution
Indrajit Singh	Platform architecture, Wazuh/Sysmon/OpenSearch integration, Streamlit dashboard development, project repository
Jay Goyal	Detection rule engineering, testing and validation, Incident Response Playbook
Vansh Gupta	AI integration, guardrails engineering, documentation and reporting

17. References and Repository

Project repository: [indrajitisingh/AlertMind: AI-Assisted Mini Security Operations Centre](#)

Supporting documents referenced in this report — the Incident Response Playbook, the MTTD & MTTR Analysis, and the system architecture and AI pipeline diagrams — are maintained as standalone files alongside this report and in the project repository.